



Too low

Too high

Correct!

GROUP TWO

A Guess Number Game

A command-line number guessing game in Python
Four levels · Algorithm · Flowchart · Full code · Presentation Q&A



Contents

- 1. Brief Run-through of the Problem 2
- 2. The Algorithm 3
 - Pseudocode — General Algorithm 3
 - Flowchart 4
 - Each Function — Pseudocode 5
- The Levels (Pseudocode) 10
 - Level 1 — Basic Version 10
 - Level 2 — Intermediate Version 10
 - Level 3 — Problem-Solving Version 11
 - Level 4 — Full Challenge Version 12
 - main() — Main Function 14
- Major Technical Concepts Used 16
- Full Code 20
- Presentation Questions & Answers 26
 - 20 Questions They May Ask Us 26
 - 10 Questions We Can Ask Them 29

1. Brief Run-through of the Problem

The project is to build a command-line number guessing game in Python.

The computer generates a secret random number within a specified range, and the player tries to guess it. After each guess, the program tells the player whether the guess is too high, too low, or correct.

The project is divided into four levels, with each level adding more functionality:

LEVEL

1

Level 1 — Basic

Generate a random number, receive one guess, compare it with the secret number, and display the result.

LEVEL

2

Level 2 — Intermediate

Allow the player to keep guessing until they are correct and track the number of attempts.

LEVEL

3

Level 3 — Problem Solving

Add input validation, difficulty levels with different ranges, and a maximum number of attempts.

LEVEL

4

Level 4 — Challenge

Add hints, scoring, star ratings, multiplayer mode, multiple rounds, and best-score tracking.

So, the project gradually develops from a simple guessing program into a complete interactive game.

2. The Algorithm

Pseudocode — General Algorithm

BEGIN

IMPORT random

DEFINE reusable function generate_secret_number()

DEFINE reusable function compare_guess()

DEFINE reusable function choose_difficulty()

DEFINE reusable function get_valid_guess()

DEFINE reusable function give_hint()

DEFINE reusable function calculate_score()

DEFINE reusable function get_rating()

DEFINE reusable function choose_game_mode()

DEFINE level1()

Run the basic number guessing game

DEFINE level2()

Run the repeated guessing game

Track the player's attempts

DEFINE level3()

Allow the player to choose difficulty

Validate the player's guesses

Limit the number of attempts

DEFINE level4()

Allow the player to choose difficulty

Allow the player to choose game mode

Provide hints

Calculate scores

Give ratings

Support multiplayer

Allow multiple rounds

Track the best score

DEFINE main()

run the game

CALL main()

END

Flowchart

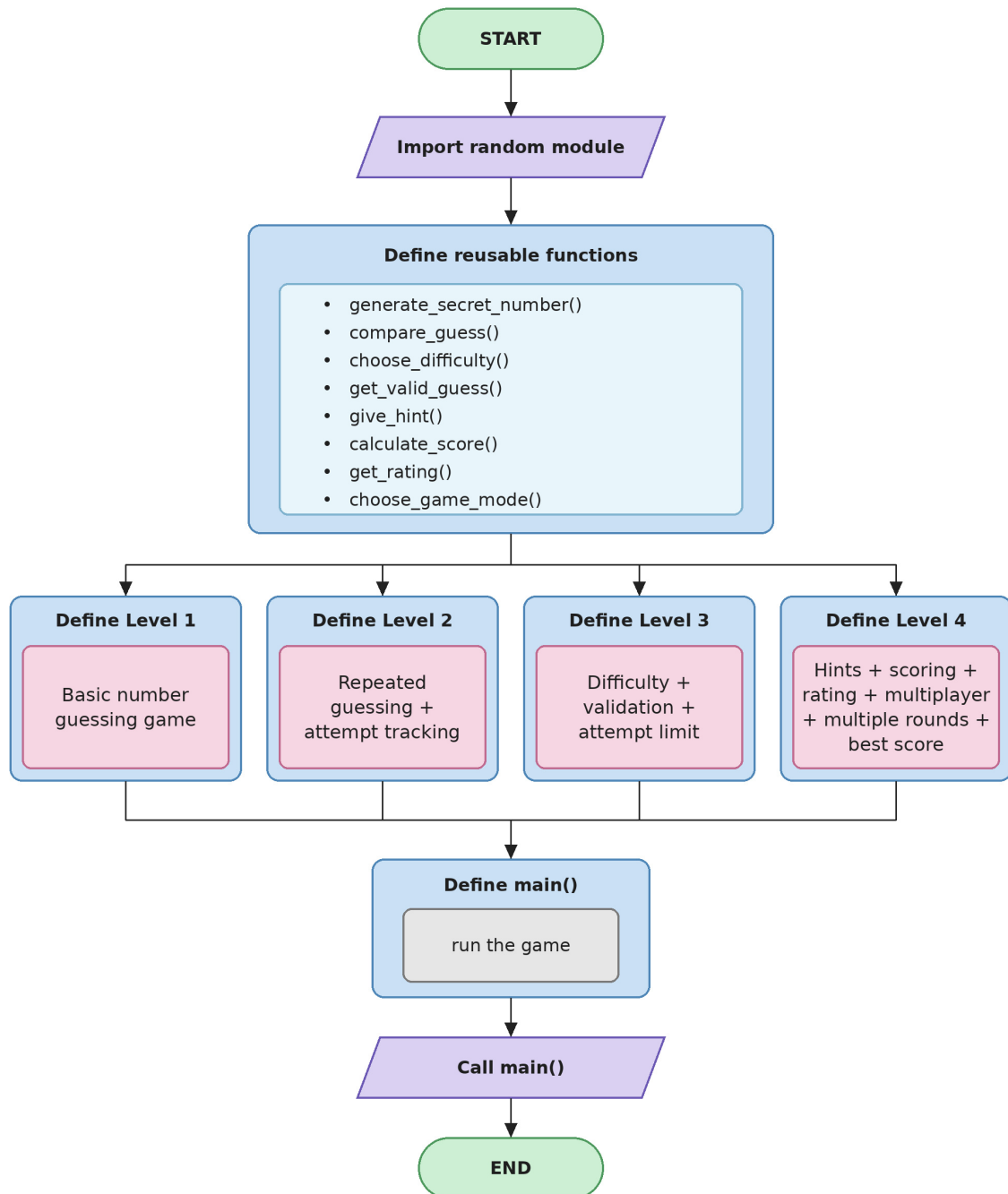


Figure 1 — Flowchart of the Guess Number Game

Each Function — Pseudocode

1 generate_secret_number(low, high)

DOES	Generates a random secret number.
PARAMETERS	<code>low, high</code> — define the number range; defaults are 1 and 100.
LOGIC	Uses <code>random.randint()</code> to generate a number within the range.
RETURNS	The generated integer.

PSEUDOCODE

```
FUNCTION generate_secret_number(low = 1, high = 100)
```

```
    Generate a random integer  
    between low and high
```

```
    Return the generated number
```

```
END FUNCTION
```

2 compare_guess(guess, secret_number)

DOES	Compares the player's guess with the secret number.
PARAMETERS	<code>guess, secret_number</code> .
LOGIC	Checks whether the guess is lower, higher, or equal.
RETURNS	<code>"Too low", "Too high", or "Correct"</code> .

PSEUDOCODE

```
FUNCTION compare_guess(guess, secret_number)
```

```
    IF guess is less than secret_number  
        Return "Too low"
```

```
    ELSE IF guess is greater than secret_number  
        Return "Too high"
```

```
    ELSE  
        Return "Correct"
```

```
END FUNCTION
```

3 `choose_difficulty()`

DOES	Lets the player select a difficulty.
PARAMETERS	None.
LOGIC	Matches the choice to a range and maximum attempts.
RETURNS	low, high, and <code>max_attempts</code> .

PSEUDOCODE

```
FUNCTION choose_difficulty()

    Display difficulty options: ( 1. Easy, 2. Medium, 3. Hard)

    Get the player's choice

    IF choice is 1
        Return low = 1, high = 50, max_attempts = 8

    ELSE IF choice is 2
        Return low = 1, high = 100, max_attempts = 10

    ELSE IF choice is 3
        Return low = 1, high = 500, max_attempts = 15

    ELSE
        Display "Invalid choice"
        Ask for the difficulty again

END FUNCTION
```

4 `get_valid_guess(low, high)`

DOES	Gets a valid guess from the player.
PARAMETERS	<code>low</code> , <code>high</code> — define the valid range.
LOGIC	Validates that the input is an integer within the range.
RETURNS	The valid integer guess.

PSEUDOCODE

```
FUNCTION get_valid_guess(low, high)

    REPEAT

        Ask the player to enter a guess

        TRY to convert the input to an integer

            IF the guess is between low and high
                Return the guess

            ELSE
                Display "Please enter a number within the range"

        IF the input cannot be converted to an integer
            Display "Invalid input"

    UNTIL a valid guess is entered

END FUNCTION
```

5

give_hint(secret_number, low, high)

DOES	Generates a hint about the secret number.
PARAMETERS	secret_number, low, high.
LOGIC	Randomly gives an even/odd or range hint.
RETURNS	The hint as a string.

PSEUDOCODE

```

FUNCTION give_hint(secret_number, low, high)

    Randomly choose between:
        Even/Odd hint
    OR
        Range hint

    IF Even/Odd hint is chosen
        IF secret_number is divisible by 2
            Return "Hint: The number is even."
        ELSE
            Return "Hint: The number is odd."

    ELSE
        Calculate midpoint = (low + high) // 2

        IF secret_number is less than or equal to midpoint
            Return a hint saying the number is between
                low and midpoint
        ELSE
            Return a hint saying the number is between
                midpoint + 1 and high

END FUNCTION

```

6 calculate_score(max_attempts, attempts)

DOES	Calculates the player's score.
PARAMETERS	max_attempts, attempts.
LOGIC	Uses $(\text{max_attempts} - \text{attempts} + 1) \times 100$.
RETURNS	The score.

PSEUDOCODE

```

FUNCTION calculate_score(max_attempts, attempts)

    Calculate score:
        score = (max_attempts - attempts + 1) × 100

    Return score

END FUNCTION

```

7 `get_rating(attempts)`

DOES	Determines the player's star rating.
PARAMETER	attempts — number of attempts used.
LOGIC	1-3 = ★★ ★, 4-6 = ★★ ★, 7+ = ★.
RETURNS	The star rating.

PSEUDOCODE

```

FUNCTION get_rating(attempts)

    IF attempts is less than or equal to 3
        Return "★★★"

    ELSE IF attempts is less than or equal to 6
        Return "★★"

    ELSE
        Return "★"

END FUNCTION

```

8 `choose_game_mode()`

DOES	Lets the player choose Single Player or Multiplayer.
PARAMETERS	None.
LOGIC	Repeats until the player enters 1 or 2.
RETURNS	The selected game mode.

PSEUDOCODE

```

FUNCTION choose_game_mode()

    REPEAT
        Display game mode options:
            1. Single Player
            2. Multiplayer

        Get the player's choice

        IF choice is 1 OR choice is 2
            Return the choice
        ELSE
            Display "Invalid game mode"
            Ask the player to choose again

    END FUNCTION

```

The Levels (Pseudocode)

1 Level 1 — Basic Version

BRIEF EXPLANATION

PURPOSE	Runs the simplest version of the number guessing game.
LOGIC	Generates a secret number, accepts one guess, compares the guess with the secret number, and displays the result.
REUSABLE FUNCTIONS USED	<code>generate_secret_number()</code> and <code>compare_guess()</code> .
KEY LIMITATION	The player gets only one guess and there is no input validation or attempt tracking.

PSEUDOCODE

```

FUNCTION level1()

    Generate a secret number

    Ask the player to enter a guess

    Compare the guess with the secret number

    Display the result

END FUNCTION

```

2 Level 2 — Intermediate Version

BRIEF EXPLANATION

PURPOSE	Allows the player to keep guessing until they find the secret number.
LOGIC	Generates a secret number, repeatedly gets guesses, counts each attempt, and stops when the guess is correct.
REUSABLE FUNCTIONS USED	<code>generate_secret_number()</code> and <code>compare_guess()</code> .
KEY ADDITION	Introduces a loop and attempt counter compared with Level 1.

PSEUDOCODE

```
FUNCTION level2()

    Generate a secret number

    Set attempts to 0

    REPEAT
        Ask the player to enter a guess

        Increase attempts by 1

        Compare the guess with the secret number

        Display the result and attempt count

        IF the guess is correct
            Display congratulations and total attempts
            Stop the loop

    END FUNCTION
```

3 Level 3 — Problem-Solving Version

BRIEF EXPLANATION

PURPOSE	Adds difficulty levels, input validation, and a maximum attempt limit.
LOGIC	The player chooses a difficulty, receives the corresponding range and attempt limit, then keeps guessing until they win or run out of attempts.
REUSABLE FUNCTIONS USED	<code>choose_difficulty()</code> , <code>generate_secret_number()</code> , <code>get_valid_guess()</code> , and <code>compare_guess()</code> .
KEY ADDITION	Handles invalid input and limited attempts.

PSEUDOCODE

```

FUNCTION level3()

    Choose difficulty

    Display maximum attempts

    Generate a secret number within the selected range

    Set attempts to 0

    WHILE attempts is less than maximum attempts

        Get a valid guess

        Increase attempts by 1

        Compare the guess with the secret number

        Display the attempt number and result

        IF the guess is correct
            Display congratulations and total attempts
            Stop the loop

    Display "Game Over"

    Display the secret number

END FUNCTION

```

4

Level 4 — Full Challenge Version

BRIEF EXPLANATION

PURPOSE	Combines all previous features and adds scoring, ratings, hints, multiplayer, and multiple rounds.
LOGIC	The player chooses difficulty and game mode, then plays either single-player or multiplayer. Hints are given after every 3 wrong guesses, scores and ratings are calculated when someone wins, and the best score is tracked across rounds.
REUSABLE FUNCTIONS USED	All 8 reusable functions.
KEY ADDITION	Introduces multiplayer, hints, scoring, ratings, best-score tracking, and replaying rounds.

PSEUDOCODE

```
FUNCTION level4()

    Set best score to 0

    REPEAT

        Choose difficulty

        Choose game mode

        IF game mode is Single Player

            Generate a secret number

            Set attempts to 0
            Set wrong guesses to 0

            WHILE attempts are less than maximum attempts

                Get a valid guess

                Increase attempts by 1

                Compare the guess with the secret number

                Display the attempt number and result

                IF the guess is correct
                    Calculate the score
                    Get the rating
                    Display score and rating

                    IF score is greater than best score
                        Update best score
                        Display "New best score"

                Stop the loop

                Increase wrong guesses by 1

                IF wrong guesses is divisible by 3
                    Give a hint

            IF maximum attempts are reached
                Display "Game Over"
                Display the secret number

        ELSE IF game mode is Multiplayer

            Generate a secret number

            Set attempts to 0
            Set wrong guesses to 0
            Set Player 1 attempts to 0
            Set Player 2 attempts to 0

            WHILE attempts are less than maximum attempts

                Player 1 makes a guess
```

```

    Increase Player 1 attempts

    Compare the guess

    IF Player 1 is correct
        Calculate score and rating
        Display Player 1 wins
        Update best score if necessary
        Stop the loop

    Player 2 makes a guess

    Increase Player 2 attempts

    Compare the guess

    IF Player 2 is correct
        Calculate score and rating
        Display Player 2 wins
        Update best score if necessary
        Stop the loop

    Increase attempts by 1

    Give a hint after every 3 wrong guesses

    IF maximum attempts are reached
        Display "Game Over"
        Display both players' attempts

    Display best score

    Ask whether the player wants another round

    Repeat until the player chooses "n"

END FUNCTION

```

5 main() — Main Function

BRIEF EXPLANATION

PURPOSE	Controls which level of the game is executed.
PARAMETERS	None.
LOGIC	Selects a level and calls the corresponding level function.
RETURN VALUE	None.
PROJECT PURPOSE	Level 4 is the final and most advanced version and would normally be the version used to run the game. Levels 1–3 are kept for presentation and learning purposes to demonstrate the progressive development of the program.

PSEUDOCODE

```
FUNCTION main()

    Select the level to demonstrate

    IF level is 1
        Run level1()

    ELSE IF level is 2
        Run level2()

    ELSE IF level is 3
        Run level3()

    ELSE
        Run level4()

END FUNCTION
```

After defining `main()`, the program starts by calling it:

```
CALL main()
```

Major Technical Concepts Used

1 random Module

The random module is a built-in Python module used to generate random values.

In this project, it is used to:

- Generate the secret number with `random.randint()`.
- Randomly select a type of hint with `random.choice()`.

2 Functional Programming Approach

The program follows a functional programming approach by breaking the solution into small functions, where each function performs a specific task.

For example:

```
generate_secret_number()
compare_guess()
calculate_score()
get_rating()
```

Each function can perform its task independently and return a result to the part of the program that called it.

3 Code Reuse

Code reuse means writing a piece of code once and using it multiple times instead of repeating the same code.

For example, `compare_guess()` is written once but can be used by Levels 1, 2, 3, and 4.

This makes the program shorter, easier to maintain, and reduces duplication.

4 Separation of Concerns

Separation of concerns means dividing a program into different parts, where each part handles a particular responsibility.

For example:

```
get_valid_guess()    → handles input validation
compare_guess()      → handles guess comparison
calculate_score()    → handles scoring
get_rating()         → handles rating
```

This prevents different responsibilities from being mixed together.

5 Single Responsibility

The Single Responsibility Principle means that a function should have one clear responsibility.

For example:

```
calculate_score() → calculates the score  
get_rating()     → determines the rating
```

Neither function tries to perform the other's job.

This makes the functions easier to understand, test, and modify.

6 Default Parameters

A default parameter is a parameter that already has a value if the caller does not provide one.

In:

```
generate_secret_number(low=1, high=100)
```

low defaults to 1 and high defaults to 100.

Therefore:

```
generate_secret_number()
```

uses the range 1-100.

But we can also provide different values:

```
generate_secret_number(1, 500)
```

7 while Loop and Control Flow

A while loop repeatedly executes a block of code as long as a condition remains true.

It is used in this project for situations where we do not know exactly how many times something needs to happen.

Examples:

- Repeatedly asking for a valid guess.
- Allowing the player to keep guessing.
- Repeating the game for another round.
- Repeatedly asking for a valid game mode.

The loop controls the flow of execution by deciding when a block should continue or stop.

8 Conditionals

Conditionals allow the program to make decisions based on conditions.

The project uses:

```
if
elif
else
```

For example, `compare_guess()` decides:

```
If guess < secret number → Too low
Else if guess > secret number → Too high
Else → Correct
```

Conditionals allow the program to respond differently depending on the situation.

9 try/except Block

A try/except block is used to handle errors without crashing the program.

In `get_valid_guess()`:

```
try:
    guess = int(input(...))
except ValueError:
    ...
```

The program tries to convert the player's input into an integer.

If the player enters something that cannot be converted, Python raises a `ValueError`. The except block catches the error and asks the player to enter a valid integer instead.

10 Unpacking

Unpacking means assigning multiple values returned from a function to multiple variables at once.

`choose_difficulty()` returns:

```
low, high, max_attempts
```

For example:

```
return 1, 100, 10
```

These values can be unpacked as:

```
low, high, max_attempts = choose_difficulty()
```

So:

```
low      → 1  
high     → 100  
max_attempts → 10
```

This makes it easy to work with multiple values returned by a function.

Full Code

```
import random

def generate_secret_number(low=1, high=100):
    """Generate and return a random number between 1 and 100."""
    rand_num = random.randint(low, high)
    print(f"randomly generated number: {rand_num}")
    return rand_num

def compare_guess(guess, secret_number):
    """Compare a player's guess with the secret number and return the result."""
    if guess < secret_number:
        return "Too low"
    elif guess > secret_number:
        return "Too high"
    else:
        return "Correct"

def choose_difficulty():
    """Ask the player to choose a difficulty and return its game settings."""
    print("Choose difficulty:")
    print("1. Easy")
    print("2. Medium")
    print("3. Hard")

    choice = input("Enter your choice: ")

    if choice == "1":
        print(f"Level: easy")
        return 1, 50, 8
    elif choice == "2":
        print(f"Level: medium")
        return 1, 100, 10
    elif choice == "3":
        print(f"Level: difficult")
        return 1, 500, 15
    else:
        print("Invalid choice.")
        return choose_difficulty()

def get_valid_guess(low, high):
    """Get and return an integer guess within the specified range."""
    while True:
        try:
            guess = int(input(f"Enter your guess ({low}-{high}): "))

            if low <= guess <= high:
                return guess

            print(f"Please enter a number between {low} and {high}.")

        except ValueError:
            print("Invalid input. Please enter an integer.")

def give_hint(secret_number, low, high):
    """Generate and return a hint about the secret number."""
    if random.choice([True, False]):
        if secret_number % 2 == 0:
            return "Hint: The number is even."
        else:
            return "Hint: The number is odd."
```

```

    else:
        midpoint = (low + high) // 2

        if secret_number <= midpoint:
            return f"Hint: The number is between {low} and {midpoint}."
        else:
            return f"Hint: The number is between {midpoint + 1} and {high}."

def calculate_score(max_attempts, attempts):
    """Calculate and return a score based on the number of attempts used."""
    return (max_attempts - attempts + 1) * 100

def get_rating(attempts):
    """Return a star rating based on the number of attempts used."""
    if attempts <= 3:
        return "★★★"
    elif attempts <= 6:
        return "★★"
    else:
        return "★"

def choose_game_mode():
    """Ask the player to choose a game mode and return the valid choice."""
    while True:
        print("\nChoose game mode:")
        print("1. Single Player")
        print("2. Multiplayer")

        mode = input("Enter your choice: ")

        if mode in ("1", "2"):
            return mode

        print("Invalid game mode. Please choose 1 or 2.")

#level1()
#→ generate secret number
#→ ask player for a guess
#→ compare guess with secret number
#→ display the result
#→ finish

def level1():
    """Run the basic version of the number guessing game."""
    secret_number = generate_secret_number()

    guess = int(input("Guess the number: "))

    result = compare_guess(guess, secret_number)

    print(result)

"""
level2()
→ generate secret number
→ set attempts = 0
→ get guess
→ increase attempts
→ compare guess
→ display result and attempt count
→ if correct → show attempts → finish
→ if wrong → ask for another guess
"""

def level2():
    """Run the number guessing game with repeated guesses and attempt tracking."""
    secret_number = generate_secret_number()

```

```

    attempts = 0

    while True:
        guess = int(input("Guess the number: "))
        attempts += 1

        result = compare_guess(guess, secret_number)

        print(f"{result}\nAttempt: {attempts}")

        if result == "Correct":
            print(f"Congratulations! You got it in {attempts} attempts.")
            break

    """
    level3()
    → choose difficulty
    → generate secret number
    → set attempts = 0
    → get valid guess
    → increase attempts
    → display attempt count
    → compare guess
    → if correct → win
    → if wrong → check maximum attempts
    → if attempts remain → repeat
    → otherwise → Game Over
    """

def level3():
    """Run the number guessing game with difficulty, validation, and attempt limits."""
    low, high, max_attempts = choose_difficulty()
    print(f"Your maximum attempts is {max_attempts}.")
    secret_number = generate_secret_number(low, high)
    attempts = 0

    while attempts < max_attempts:
        guess = get_valid_guess(low, high)
        attempts += 1

        print(f"Attempt: {attempts}")

        result = compare_guess(guess, secret_number)
        print(result)

        if result == "Correct":
            print(f"Congratulations! You got it in {attempts} attempts.")
            break

    print("Game Over!")
    print(f"The secret number was {secret_number}.")

    """
    level4()
    → choose difficulty
    → choose game mode
    → play the game
    → give hints after every 3 wrong guesses
    → calculate score
    → show rating
    → update best score
    → ask for another round
    → finish
    """

def level4():
    """Run the full challenge version of the number guessing game."""

```

```
best_score = 0

while True:
    low, high, max_attempts = choose_difficulty()
    mode = choose_game_mode()

    if mode == "1":
        secret_number = generate_secret_number(low, high)
        attempts = 0
        wrong_guesses = 0

        while attempts < max_attempts:
            guess = get_valid_guess(low, high)
            attempts += 1

            print(f"Attempt: {attempts}")

            result = compare_guess(guess, secret_number)
            print(result)

            if result == "Correct":
                score = calculate_score(max_attempts, attempts)
                rating = get_rating(attempts)

                print(f"Congratulations! You scored {score} points.")
                print(f"Rating: {rating}")

                if score > best_score:
                    best_score = score
                    print("New best score!")

                break

            wrong_guesses += 1

            if wrong_guesses % 3 == 0:
                print(give_hint(secret_number, low, high))

        print("Game Over!")
        print(f"The secret number was {secret_number}.")

    elif mode == "2":
        secret_number = generate_secret_number(low, high)

        attempts = 0
        wrong_guesses = 0
        player1_attempts = 0
        player2_attempts = 0

        while attempts < max_attempts:

            # Player 1
            print("\nPlayer 1's turn")
            guess = get_valid_guess(low, high)
            player1_attempts += 1

            result = compare_guess(guess, secret_number)
            print(result)

            if result == "Correct":
                score = calculate_score(max_attempts, attempts + 1)
                rating = get_rating(attempts + 1)

                print("Player 1 wins!")
                print(f"Attempts: {player1_attempts}")
                print(f"Score: {score}")
                print(f"Rating: {rating}")
```

```
        if score > best_score:
            best_score = score
            print("New best score!")

        break

    wrong_guesses += 1

    if wrong_guesses % 3 == 0:
        print(give_hint(secret_number, low, high))

    # Player 2
    print("\nPlayer 2's turn")
    guess = get_valid_guess(low, high)
    player2_attempts += 1

    result = compare_guess(guess, secret_number)
    print(result)

    if result == "Correct":
        score = calculate_score(max_attempts, attempts + 1)
        rating = get_rating(attempts + 1)

        print("Player 2 wins!")
        print(f"Attempts: {player2_attempts}")
        print(f"Score: {score}")
        print(f"Rating: {rating}")

        if score > best_score:
            best_score = score
            print("New best score!")

        break

    wrong_guesses += 1

    if wrong_guesses % 3 == 0:
        print(give_hint(secret_number, low, high))

    # One complete attempt consists of both players guessing.
    attempts += 1

    print("\nGame Over!")
    print(f"The secret number was {secret_number}.")
    print(f"Player 1 attempts: {player1_attempts}")
    print(f"Player 2 attempts: {player2_attempts}")

    print(f"\nBest score: {best_score}")

    # Validate play-again choice
    while True:
        play_again = input("Play another round? (y/n): ").lower()

        if play_again in ("y", "n"):
            break

        print("Invalid choice. Please enter y or n.")

    if play_again == "n":
        break

def main():
    level = 4
    if level == 1:
        levell()
    elif level == 2:
```

```
    level2()
elif level == 3:
    level3()
else:
    level4()

#level4()

main()
```

Presentation Questions & Answers

20 Questions They May Ask Us

1**What is the purpose of the project?****ANSWER**

To build a number guessing game that progressively introduces input validation, difficulty levels, hints, scoring, ratings, multiplayer, and multiple rounds.

2**Why did you divide the program into functions?****ANSWER**

To make the program modular, reusable, easier to understand, test, and maintain.

3**What programming paradigm did you use?****ANSWER**

A functional programming approach, where the program is divided into functions that perform specific tasks and return results.

4**Why didn't you use OOP?****ANSWER**

The project is small enough to be handled effectively with functions. OOP could be used, but it would introduce classes and objects that are not necessary for the current requirements.

5**Why doesn't Level 2 call Level 1?****ANSWER**

The levels are independent stages of the solution. Code reuse is achieved through the smaller reusable functions rather than making one level dependent on another.

6**Why doesn't Level 4 call Level 3?****ANSWER**

To keep each level independent and prevent changes in one level from affecting another. Level 4 combines the required features independently.

7**Why are Levels 1–3 kept if Level 4 is the final version?****ANSWER**

Because this is a presentation and learning project. Keeping all four levels demonstrates the progressive development from the basic solution to the advanced solution.

8 What is code reuse in your project?

ANSWER

Functions such as `compare_guess()` and `get_valid_guess()` are written once and reused by different levels.

9 What is separation of concerns?

ANSWER

It means dividing the program so that different parts handle different responsibilities. For example, `calculate_score()` handles scoring while `get_rating()` handles ratings.

10 What is the Single Responsibility Principle?

ANSWER

It means that a function should have one clear responsibility. For example, `compare_guess()` only compares the guess with the secret number.

11 Why did you use the random module?

ANSWER

To generate the secret number randomly and to randomly select the type of hint given to the player.

12 What is a default parameter?

ANSWER

It is a parameter with a predefined value. In `generate_secret_number()`, `low=1` and `high=100` are default values.

13 Why did you use a while loop?

ANSWER

Because we don't know beforehand how many guesses the player will need. The loop continues until the player wins, reaches the attempt limit, or chooses to stop.

14 Why did you use try/except?

ANSWER

To handle invalid input without crashing the program. For example, entering text where an integer is expected causes a `ValueError`.

15 What is unpacking in your program?

ANSWER

It is assigning multiple returned values to multiple variables, such as: `low, high, max_attempts = choose_difficulty()`.

16 How does the scoring system work?

ANSWER

The formula is $(\text{max_attempts} - \text{attempts} + 1) \times 100$. The maximum score is 1000, and the score decreases as more attempts are used.

17 How does the hint system work?

ANSWER

After every three wrong guesses, the program randomly provides either an even/odd hint or a range hint.

18 How does multiplayer work?

ANSWER

Both players take turns guessing the same secret number. Their attempts are tracked separately, and the first player to guess correctly wins.

19 Why do you use return instead of `print()` in some functions?

ANSWER

`return` sends a result back to the calling code, allowing the result to be used elsewhere. `print()` only displays information.

20 If you had more time, how would you improve the program?

ANSWER

I could add automated tests, a graphical interface, persistent score storage, a leaderboard, and potentially redesign it using OOP if the application became much larger.

10 Questions We Can Ask Them

These are questions we can ask based directly on concepts demonstrated in our code.

1 Why is modular programming useful in software development?

EXPECTED ANSWER

It divides a large problem into smaller, manageable components that are easier to maintain, test, and reuse.

2 What is the difference between a parameter and an argument?

EXPECTED ANSWER

A parameter is the variable defined in a function, while an argument is the actual value passed to that function.

3 What is the difference between return and `print()`?

EXPECTED ANSWER

return sends a value back to the caller, while `print()` displays a value to the user.

4 Why is input validation important?

EXPECTED ANSWER

It prevents invalid data from causing incorrect program behaviour or errors.

5 What is the purpose of try/except?

EXPECTED ANSWER

It allows a program to handle exceptions and continue running instead of crashing.

6 Why are loops useful in programming?

EXPECTED ANSWER

They allow a block of code to be executed repeatedly while a condition is satisfied.

7 What is the benefit of using default parameters?

EXPECTED ANSWER

They provide fallback values, making a function easier to call while still allowing custom values.

8 What is code reuse?

EXPECTED ANSWER

Using existing code, such as a function, in multiple places instead of rewriting the same logic.

9

What is the difference between functional programming and OOP?**EXPECTED ANSWER**

Functional programming focuses on functions and operations, while OOP organises programs around objects containing data and behaviour.

10

Why is separation of concerns important?**EXPECTED ANSWER**

It keeps different responsibilities separate, making the program easier to understand, maintain, test, and modify.